

Intensity Transformation on Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.sanitize'] = True
```

```
print(np.e)
```

```
2.718281828459045
```

Image negatives

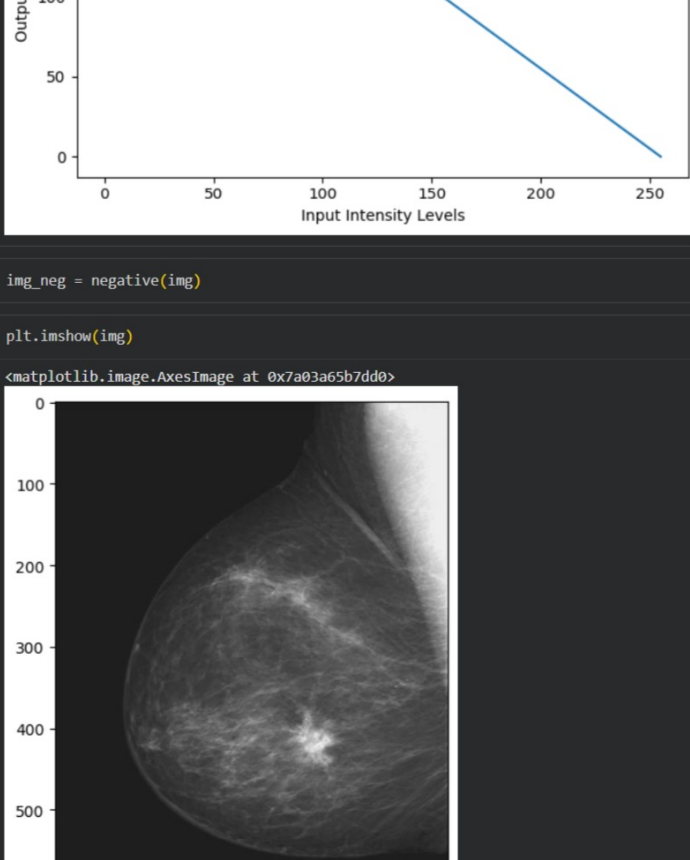
```
img = cv2.imread('fig004(a)(breast_digital_xray).tif')
```

```
def negative(r):
    s = 255 - r
    return s
```

```
x_values = np.linspace(0, 255, 500)
y_values = negative(x_values)
```

```
plt.plot(x_values, y_values)
plt.xlabel('Input Intensity Levels')
plt.ylabel('Output Intensity Levels')
```

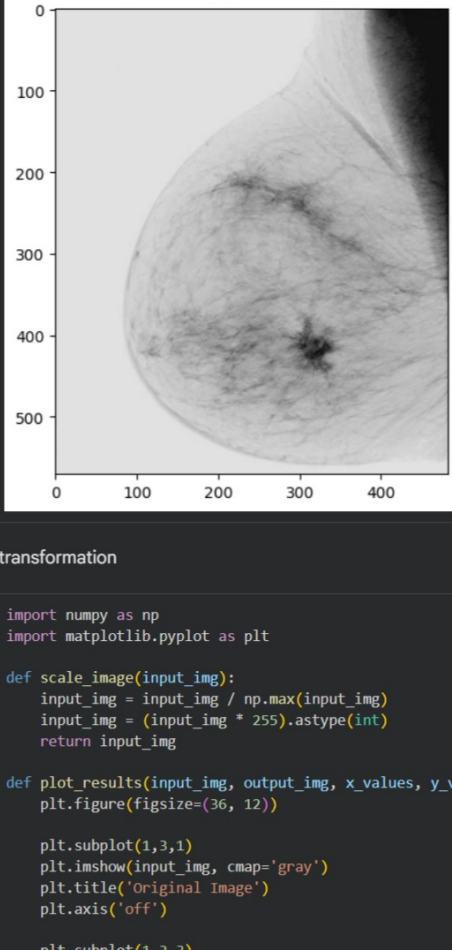
```
text(0, 0.5, 'Output Intensity Levels')
```



```
img_neg = negative(img)
```

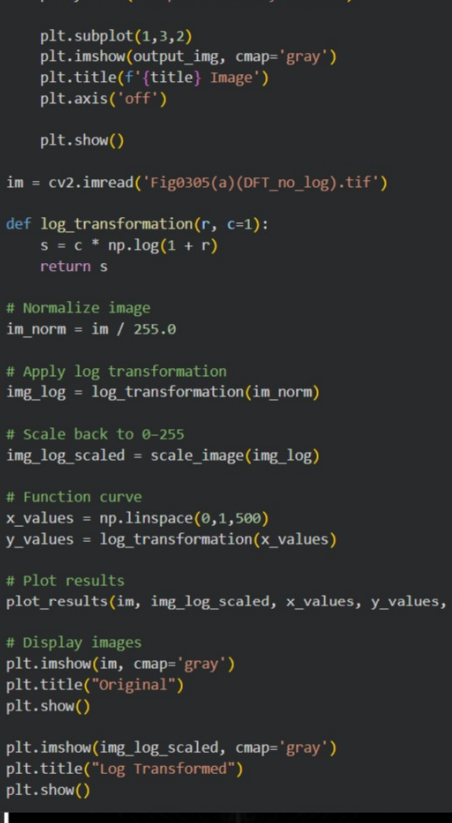
```
plt.imshow(img)
```

```
output[10, image, axis=image at 0x2a1bdc07b1d8]
```



```
plt.imshow(img_neg)
```

```
output[10, image, axis=image at 0x2a1bdc07b1d8]
```



Log transformation

```
import numpy as np
import matplotlib.pyplot as plt

def scale_image(input_img):
    input_img = input_img / np.max(input_img)
    input_img = (input_img * 255).astype(int)
    return input_img

def plot_results(input_img, output_img, x_values, y_values, title):
    plt.figure(figsize=(8, 12))

    plt.subplot(1,3,1)
    plt.imshow(input_img, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1,3,2)
    plt.plot(x_values, y_values)
    plt.title('Title function')
    plt.xlabel('Input Intensity Levels')
    plt.ylabel('Output Intensity Levels')

    plt.subplot(1,3,3)
    plt.imshow(output_img, cmap='gray')
    plt.title('Title Image')
    plt.axis('off')

    plt.show()
```

```
im = cv2.imread('fig005(a)(DT_no_log).tif')
```

```
def log_transformation(r, c=1):
    s = c * (r * np.log(r + 1))
    return s
```

```
# Normalize image
im_norm = im / 255.0
```

```
# Apply log transformation
img_log = log_transformation(im_norm)
```

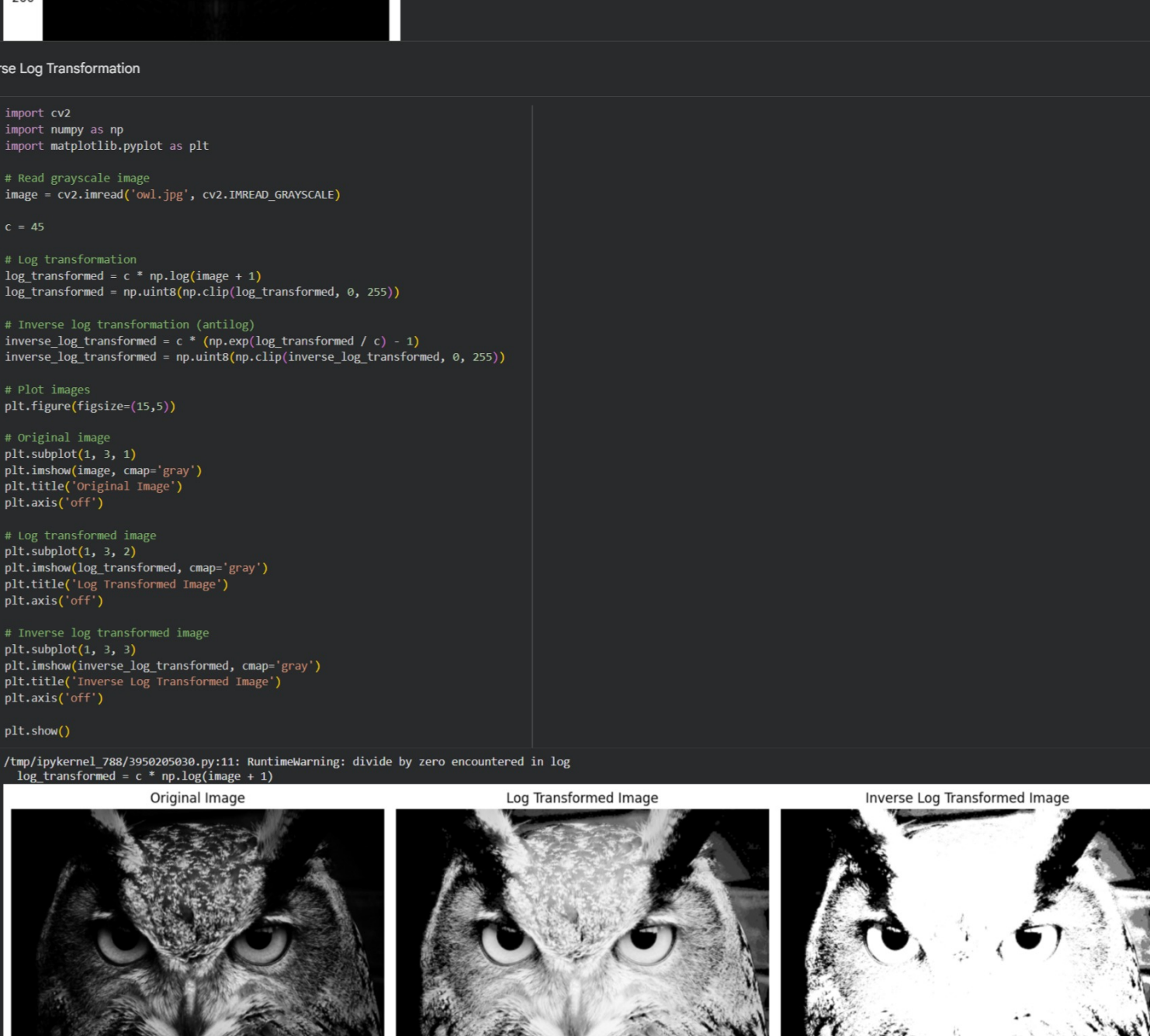
```
# Scale back to 0-255
img_log_scaled = scale_image(img_log)
```

```
# function curve
x_values = np.linspace(0,1,500)
y_values = log_transformation(x_values)
```

```
# Plot results
plot_results(im, img_log_scaled, x_values, y_values, 'log transformation')
```

```
# Display images
plt.imshow(im, cmap='gray')
plt.title('Original')
plt.show()
```

```
plt.imshow(img_log_scaled, cmap='gray')
plt.title('log transformed')
plt.show()
```



Inverse Log Transformation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
image = cv2.imread('owl.jpg', cv2.IMREAD_GRAYSCALE)

c = 45
```

```
# log transformation
log_transformed = c * np.log(image + 1)
log_transformed = np.uint8(np.clip(log_transformed, 0, 255))
```

```
# Inverse log transformation (anti-log)
inverse_log_transformed = c * (np.exp(log_transformed / c) - 1)
inverse_log_transformed = np.uint8(np.clip(inverse_log_transformed, 0, 255))
```

```
# Plot images
plt.figure(figsize=(15,5))
```

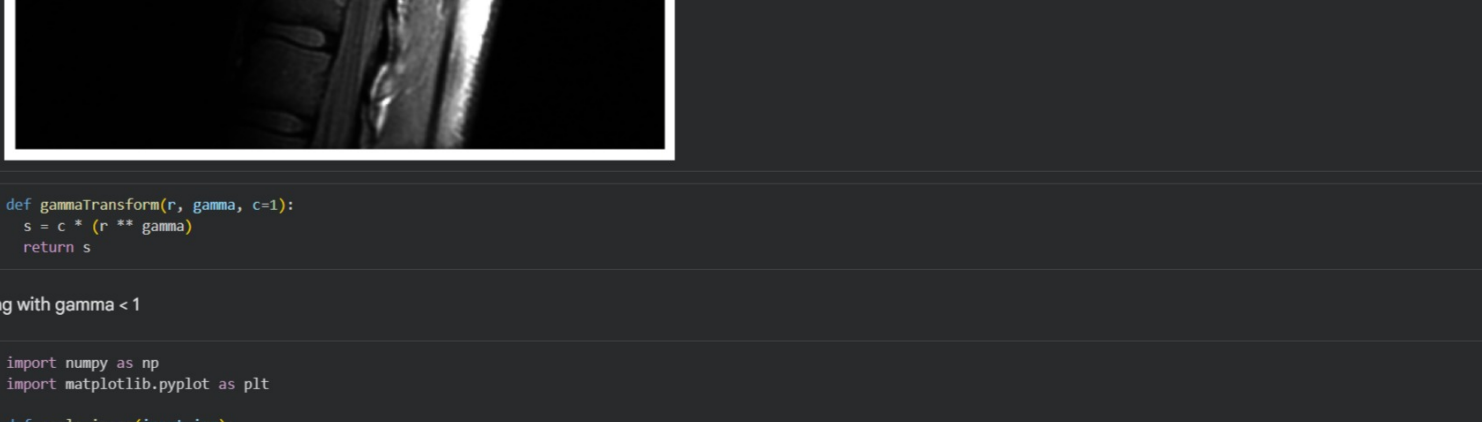
```
# Original image
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')
```

```
# log transformed image
plt.subplot(1, 3, 2)
plt.imshow(log_transformed, cmap='gray')
plt.title('log transformed image')
plt.axis('off')
```

```
# Inverse log transformed image
plt.subplot(1, 3, 3)
plt.imshow(inverse_log_transformed, cmap='gray')
plt.title('Inverse log transformed image')
plt.axis('off')
```

```
plt.show()
```

```
/tmp/ipykernel_788/392090838.py:11: RuntimeWarning: divide by zero encountered in log
log_transformed = c * np.log(image + 1)
```



Power law (Gamma) Transformation

```
spine_img = cv2.imread('fig030(a)(fractured_spine).tif')
```

```
plt.figure(figsize=(12, 8))
plt.imshow(spine_img)
plt.axis('off')
```

```
(np.float64( 0.5), np.float64(745.5), np.float64(475.5), np.float64( 0.5))
```



```
def gammaTransform(r, gamma, c=1):
    s = c * (r ** gamma)
    return s
```

Trying with gamma < 1

```
import numpy as np
import matplotlib.pyplot as plt

def scale_image(input_img):
    input_img = input_img / np.max(input_img)
    input_img = (input_img * 255).astype(int)
    return input_img
```

```
img_gamma = gammaTransform(spine_img, 0.4)
img_gamma_scaled = scale_image(img_gamma)
```

```
# require for plotting
x_values = np.linspace(0, 255, 500)
y_values = log_transformation(x_values, 0.4)
```

```
plot_results(spine_img, img_gamma_scaled, x_values, y_values, 'gamma 0.4')
```



```
gamma_values = [0.6, 0.4, 0.3]
```

```
for gamma in gamma_values:
    print(f'Applying Gamma Transformation with gamma = {gamma}')
    img_gamma = gammaTransform(spine_img, gamma)
    img_gamma_scaled = scale_image(img_gamma)
```

```
# require for plotting
x_values_gamma = np.linspace(0, 255, 500)
y_values_gamma = gammaTransform(x_values_gamma, gamma)
```

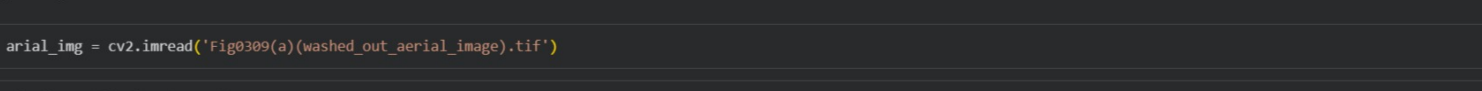
```
plot_results(spine_img, img_gamma_scaled, x_values_gamma, y_values_gamma, f'gamma {gamma}')
```



```
Applying Gamma Transformation with gamma = 0.4
```



```
Applying Gamma Transformation with gamma = 0.3
```



Trying with gamma > 1

```
arial_img = cv2.imread('fig030(b)(washed_out_aerial_image).tif')
```

```
plt.figure(figsize=(10, 6))
plt.imshow(arial_img)
plt.axis('off')
```

```
(np.float64( 0.5), np.float64(764.5), np.float64(420.5), np.float64( 0.5))
```



```
gamma_values = [0.6, 0.4, 0.3]
```

```
for gamma in gamma_values:
    print(f'Applying Gamma Transformation with gamma = {gamma}')
    img_gamma = gammaTransform(arial_img, gamma)
    img_gamma_scaled = scale_image(img_gamma)
```

```
# require for plotting
x_values_gamma = np.linspace(0, 255, 500)
y_values_gamma = gammaTransform(x_values_gamma, gamma)
```

```
plot_results(arial_img, img_gamma_scaled, x_values_gamma, y_values_gamma, f'gamma {gamma}')
```



```
Applying Gamma Transformation with gamma = 0.4
```



```
Applying Gamma Transformation with gamma = 0.6
```



```
Applying Gamma Transformation with gamma = 3.0
```

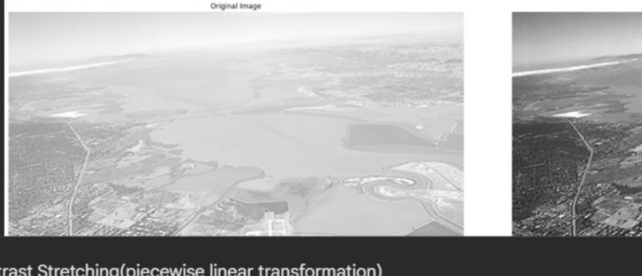


Contrast Stretching (piecewise linear transformation)

```
im = cv2.imread('fig010(b)(washed_out_pollen_image).tif')
```

```
plt.figure(figsize=(10, 6))
plt.imshow(im)
plt.axis('off')
```

```
(np.float64( 0.5), np.float64(888.5), np.float64(420.5), np.float64( 0.5))
```



```
def piecewiseLinear(r, r1, s1, r2, s2):
    if r < r1:
        s = (s1 / r1) * r
    elif r > r1 and r < r2:
        s = ((s2 - s1) / (r2 - r1)) * (r - r1) + s1
    else:
        s = ((s2 - s2) / (r25 - r2)) * (r1 - r2) + s2
    return int(s)
```

```
piecewiseLinearVec = np.vectorize(piecewiseLinear)
```

```
x_values = np.linspace(0, 255, 500)
y_values = piecewiseLinearVec(x_values, 80, 30, 150, 100)
```

```
transformed_im = piecewiseLinearVec(im, 80, 30, 150, 100)
```

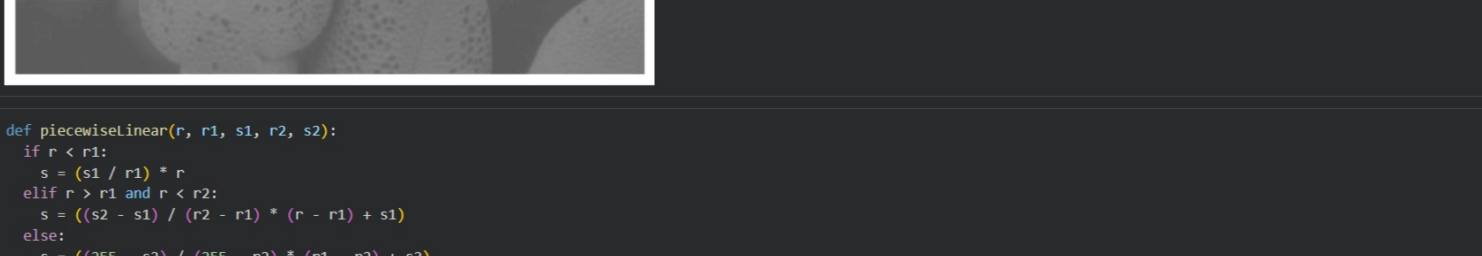
```
plot_results(im, transformed_im, x_values, y_values, 'piecewise1')
```



```
mean = np.mean(im)
transformed_im2 = piecewiseLinearVec(im, mean, 0, mean, 255)
```

```
x_values = np.linspace(0, 255, 500)
y_values = piecewiseLinearVec(x_values, mean, 0, mean, 255)
```

```
plot_results(im, transformed_im2, x_values, y_values, 'piecewise2')
```



Intensity Level Slicing

```
kidney_im = cv2.imread('fig012(a)(kidney).tif')
```

```
plt.figure(figsize=(12, 8))
plt.imshow(kidney_im)
plt.axis('off')
```

```
(np.float64( 0.5), np.float64(719.5), np.float64(420.5), np.float64( 0.5))
```



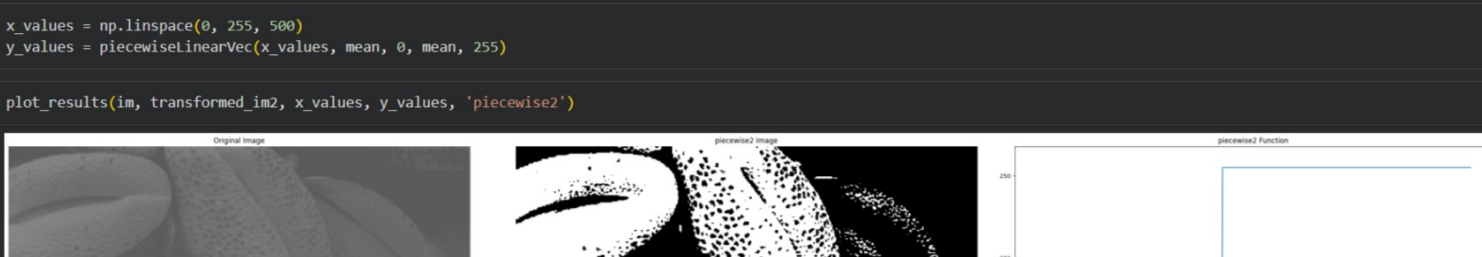
```
def intensityLevelTransform(r):
    if r > 140 and r < 210:
        return 255
    else:
        return 0
```

```
intensityLevelTransformVec = np.vectorize(intensityLevelTransform)
```

```
transform1 = intensityLevelTransformVec(kidney_im)
```

```
x_values = np.linspace(0, 255, 500)
y_values = intensityLevelTransformVec(x_values)
```

```
plot_results(kidney_im, transform1, x_values, y_values, 'intensitylevel1')
```



```
def intensityLevelTransform2(r):
    if r > 140 and r < 210:
        return 255
    else:
        return r
```

```
intensityLevelTransform2Vec = np.vectorize(intensityLevelTransform2)
```

```
transform2 = intensityLevelTransform2Vec(kidney_im)
```

```
x_values = np.linspace(0, 255, 500)
y_values = intensityLevelTransform2Vec(x_values)
```

```
plot_results(kidney_im, transform2, x_values, y_values, 'intensitylevel2')
```

